MIPHI · POWERED BY PHISON

Vane with aiDAPTIV+

An AI research and web-search assistant that runs on your own hardware and remembers what it has already read.

User Manual and Setup Guide

01
Why it matters
What Vane and aiDAPTIV+ do, and what your team gets out of them.

02
Setting it up
Step by step instructions for installing Vane and connecting it to aiDAPTIV+.

03
Checking it works
How to confirm on your own system that aiDAPTIV+ is saving time.

Version 1.0 · October 2026 · Prepared by MiPhi

Contents

Why Vane with aiDAPTIV+.....3

A typical example.....3
What Vane is.....3
What aiDAPTIV+ adds.....3
What happens when you ask Vane a question.....4
Benefits.....4
Test results.....5
Where it helps most.....5
How the setup is organised.....6

Part 1: Before You Begin.....7

What you need.....7
Start the model server.....7
Install Node.js.....8
Cloning the Vane repository.....8

Part 2: Installing and Starting Vane.....9

Install the dependencies.....9
Rebuild the database driver.....10
Start Vane.....10

Part 3: Connecting Vane to aiDAPTIV+.....11

Add an aiDAPTIV connection.....11
Add the chat model.....12
Choose the chat and embedding models.....13
Restart Vane.....14

Part 4: Using Vane.....15

The Vane screen.....15
Choose where Vane searches.....15
Chat with a document.....16

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working.....18

Check the cache folder on disk.....18
Check the cache is resumed after a restart.....18
Confirm a cache hit.....18

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit.....19

What to measure.....19
Cache hit (KV reuse) results.....19
Benchmark your own system.....20

Part 7: Troubleshooting.....21

Fixing the better-sqlite3 error.....21
Installation checklist.....23

Why Vane with aiDAPTIV+

Most teams would like an AI assistant that can search the web, read their documents and give them a clear answer with sources. Two things usually hold them back. The first is privacy: the well-known AI search services run in someone else's cloud, so every question and every document has to be sent there first. The second is speed: give a local AI model a long document and you can wait minutes before the first word of the answer appears, and every follow-up question can start that wait all over again.

Vane with aiDAPTIV+ was put together to deal with both. This chapter explains how, in plain terms. If you would rather go straight to the installation, skip ahead to the Setup Guide.

A typical example

Say an engineer attaches a long product manual to Vane and asks, “explain this?”. The AI model has to read the whole manual before it can write the first word of the answer, and on a local PC that first read is where most of the waiting happens.

Then the engineer asks a follow-up question about the same manual. Without aiDAPTIV+, the model reads all of it again and the wait starts over.

With aiDAPTIV+, the model keeps what it learned the first time. The follow-up question can reuse that work, so the answer starts much sooner, even if the model server has been restarted in between. And the manual never leaves your own machine.

What Vane is

Vane is an open-source, AI-powered answering engine: you ask a question in plain language, and Vane searches the web, academic sources, discussion sites or your own uploaded files, then writes an answer with the sources it used. It runs in your web browser at `http://localhost:3000`, and it can use an AI model running on your own hardware instead of a cloud service.

Web search, with sources
Vane searches the web through SearxNG, a private metasearch engine, and lists the sources next to every answer.

Academic and social search
Switch on Academic to search research sites such as arXiv, or Social to search discussion sites such as Reddit.

Chat with your files
Attach a PDF or other document and ask questions about it. Vane reads it and quotes it as a source.

Bring your own model
Connect Vane to aiDAPTIV+, Ollama, LM Studio, OpenAI and others from the settings screen.

What aiDAPTIV+ adds

When an AI model reads a document or a set of search results, it builds up a set of working notes about the text. Engineers call these notes the **KV cache**. They are normally kept in the graphics card's memory, which is small and expensive, so they get thrown away when space runs out or the server restarts. The next request then has to start from the beginning.

aiDAPTIV+, made by Phison, keeps those notes on a fast SSD instead. There is far more room there, and nothing gets thrown away. When a new request involves material the model has already read, it loads its notes and carries on.

> **A SIMPLE WAY TO THINK ABOUT IT**
>
> Picture a librarian who answers questions about a book. Without aiDAPTIV+, they read the whole book again every time someone asks something. With aiDAPTIV+, they keep their notes on the shelf and can answer the next person almost straight away, even if the library closed overnight in between.

What happens when you ask Vane a question

Your team never needs to think about any of this, but if you are curious, this is the sequence:

1
You ask a question
You type a question in Vane, choose the sources (Web, Academic, Social) or attach a file.

→

2
Vane gathers the material
Vane searches the chosen sources, reads your files and sends everything to the AI model on your own hardware.

→

3
aiDAPTIV+ checks its notes
Before reading anything again, aiDAPTIV+ looks on the SSD for notes it already has.

→

4
The answer comes back
Vane shows the answer with its sources, and any new notes are saved for next time.

Benefits

Benefit	In practice
Your data stays in-house	The model, your documents and your questions stay on your own hardware. Nothing is sent to an outside AI provider.
Faster follow-up questions	Asking more about the same document reuses earlier work, so the model starts answering in seconds rather than minutes.
Survives restarts	The notes sit on the SSD rather than in temporary memory, so the model still remembers what it has read after the server restarts.
Better use of existing hardware	Long documents and long search results no longer have to fit in the graphics card's memory, because the SSD takes the overflow.
Predictable cost	There are no per-question or per-token cloud charges.
Answers you can check	Every answer lists its sources, whether they are web pages, papers, forum threads or your own files.
Familiar web interface	Vane runs in the browser, with a home page, a Discover feed and a Library of past conversations.
Set up in minutes	aiDAPTIV is a built-in connection type in Vane. You enter the server address once and choose your model.

Test results

We asked Vane about a document of about 13,400 tokens, restarted the model server, and then asked again. On the second run aiDAPTIV+ loaded almost the whole prompt from its notes on the SSD instead of reading it again. Parts 5 and 6 of this guide show you how to check the same things on your own system.

1.1 s

Time to the first word on a 13,400-token prompt after a server restart, with aiDAPTIV+ reusing its saved notes.

13,392

Tokens loaded from the aiDAPTIV+ SSD cache for that request. Only 6 new tokens had to be read.

2 of 2

Requests Vane sent for the question that were served from the cache, both with over 12,900 cached tokens.

Measured by MiPhi in October 2026 using the gemma-4-26B-A4B model (4-bit) on an Intel Core Ultra 7 356H PC with integrated graphics and 32 GB of memory, with the aiDAPTIV+ middleware and an NVMe SSD cache. Your results will depend on your hardware, the model and the size of your documents.

WHAT THIS MEANS DAY TO DAY

In our CrewAI tests on the same model and the same class of PC, a prompt of this size kept the user waiting about **4 minutes 38 seconds** for the first word without aiDAPTIV+. With aiDAPTIV+, Vane started answering in just over a second, and the saved notes were still there after the server restarted.

Where it helps most

Research and analysis teams ask Vane question after question about the same long reports, papers and market studies.

Engineering and R&D teams attach specifications, design documents and test reports that are too sensitive for a public cloud.

Support and service teams look up answers in product manuals and troubleshooting guides.

Legal and compliance teams check contracts and policies without the text ever leaving the company.

Anyone who researches on the web gets a cited summary of many pages instead of opening them one by one, from a model that runs on their own PC.

How the setup is organised

The Setup Guide takes you from a fresh machine to your first answer. It is split into seven parts:

Part	What you do	Result
Part 1 **Before You Begin**	Check the requirements, start the AI model server, install Node.js and download Vane.	Everything Vane needs is in place.
Part 2 **Installing and Starting Vane**	Install the dependencies and start Vane.	Vane open in your browser.
Part 3 **Connecting Vane to aiDAPTIV+**	Add an aiDAPTIV connection, a chat model and an embedding model.	Vane answering with your own model.
Part 4 **Using Vane**	Search the web, academic and social sources, and chat with your documents.	Your first answers, with sources.
Part 5 **Verifying the Cache**	Look at the SSD cache folder and the server log.	Confirmation that aiDAPTIV+ is saving and reusing its notes.
Part 6 **Benchmark**	Check the time to first token and the token speed when saved work is reused.	Figures you can share with your team.
Part 7 **Troubleshooting**	Look up a message you see during setup.	A fix for the most common problems.

When you are ready, turn the page and start with Part 1.

SETUP GUIDE

Getting it running

Vane is a Node.js web application that you run on your own PC and open in a browser. It connects to an AI model that is served separately by the aiDAPTIV+ middleware. The commands in this guide are for Windows PowerShell, which is what we used for the screenshots.

Part 1: Before You Begin

What you need

Requirement	Details
Windows PC	Windows 10 or 11. The commands in this guide are for PowerShell.
aiDAPTIV+ middleware	Installed as per its own setup guide, with the aiDAPTIV NVMe cache path configured within the middleware.
AI model server	The aiDAPTIV llama.cpp server, started through the middleware and serving your model (a .gguf file). By default it listens on `http://localhost:8080/v1`.
Node.js and npm	Node.js 24 (tested with v24.21.0 and npm 11.19.0) or Node.js 22. We recommend installing it with NVM, the Node Version Manager, so you can switch versions if needed.
Git	Used to download our Vane repository (miphi-dev branch). You can also download it from GitHub as a ZIP file.
SearxNG (for web search)	Vane's Web, Academic and Social sources search through a SearxNG instance. You can still chat with your own files without it.
Internet connection	Needed to download Vane and its packages, for web search, and the first time the default embedding model is downloaded.

Start the model server

Start the AI model server through the aiDAPTIV+ middleware and confirm that it reports a listening, OpenAI-compatible address, for example `http://localhost:8080/v1`. You can check it from a terminal:

```
curl http://localhost:8080/v1/models
```

The reply lists the model the server is serving, for example `gemma-4-26B-A4B-it-UD-Q4_K_M.gguf`. Keep a note of this name, as you will add it to Vane in Part 3.

> **NOTE**
> Vane does not load the model itself. It talks to whatever OpenAI-compatible address your aiDAPTIV+ middleware exposes, so get that server running and confirmed first.

Install Node.js

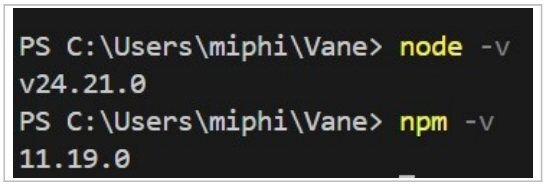
Vane needs Node.js and npm. We recommend installing Node.js through **NVM (Node Version Manager)**. NVM makes it easy to switch between Node.js 24 and Node.js 22, which is the quickest fix if you run into the native-module problem described in Part 7.

1. Install NVM for Windows, then install and select Node.js 24:

```
nvm install 24
nvm use 24
```

2. Open a new PowerShell window and check the versions:

```
node -v
npm -v
```



Node.js v24.21.0 and npm 11.19.0, the versions used in this guide

Cloning the Vane repository

Use our Vane repository. The **miphi-dev** branch includes the built-in aiDAPTIV connection type that Part 3 uses.

github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-Vane (miphi-dev branch)

1. Open PowerShell, then clone the miphi-dev branch into a folder named vane and switch into it:

```
git clone -b miphi-dev https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-
Integration-Vane.git vane
cd vane
```

2. All the commands in Parts 2 to 7 are run from this vane folder. If you leave out the folder name at the end of the clone command, Git names the folder aiDAPTIV-Integration-Vane instead.

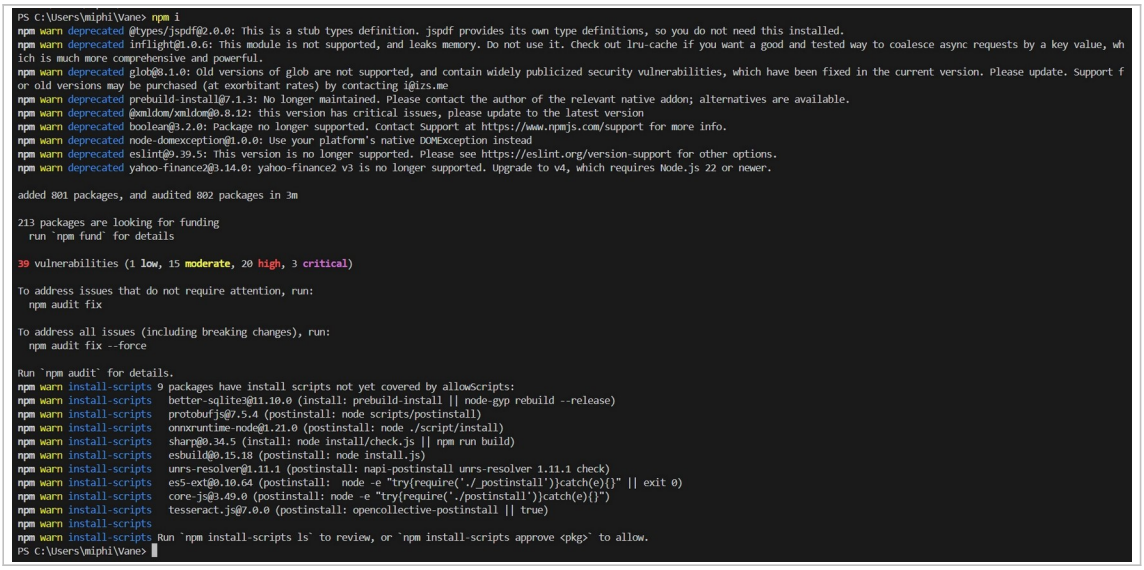
Part 2: Installing and Starting Vane

Install the dependencies

1. In the vane folder, install Vane's packages. This takes a few minutes the first time:

```
npm i
```

2. npm prints a list of warnings while it works. At the end you should see a line such as “added 801 packages”.



npm i: the packages are installed, followed by deprecation and install-script warnings

Most of these warnings are normal and do not stop Vane from working:

Warning	What it means
npm warn deprecated …	Some packages Vane depends on have newer versions. This is informational.
… vulnerabilities (… low, … moderate, … high, … critical)	npm's security audit of the dependencies. Do not run `npm audit fix --force`, as it can upgrade packages to versions Vane does not support.
npm warn install-scripts … not yet covered by allowScripts	Newer versions of npm block packages from running their install scripts until you approve them. One of these packages, better-sqlite3, builds the database driver Vane needs, so it must be allowed to run (see below).

Rebuild the database driver

Vane stores its settings and chats in a local SQLite database through the **better-sqlite3** package. This package contains a native module that has to match your exact Node.js version. Rebuilding it after `npm i` makes sure it does:

```
npm rebuild better-sqlite3
```

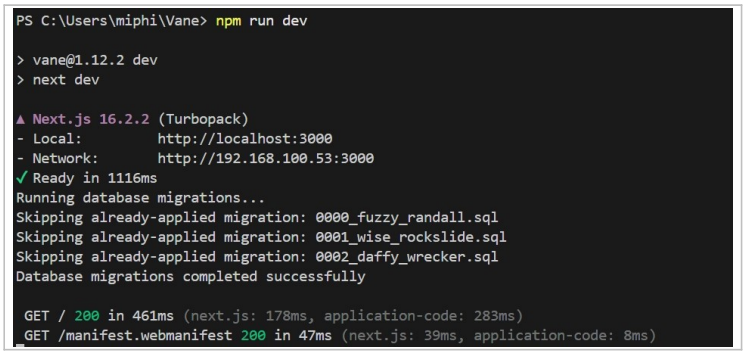
If npm reports that the install script is blocked, run `npm install-scripts ls` to review the blocked packages and approve better-sqlite3 as npm suggests, then rebuild again. If you see an error mentioning `NODE_MODULE_VERSION`, follow the steps in Part 7.

Start Vane

1. Start the development server:

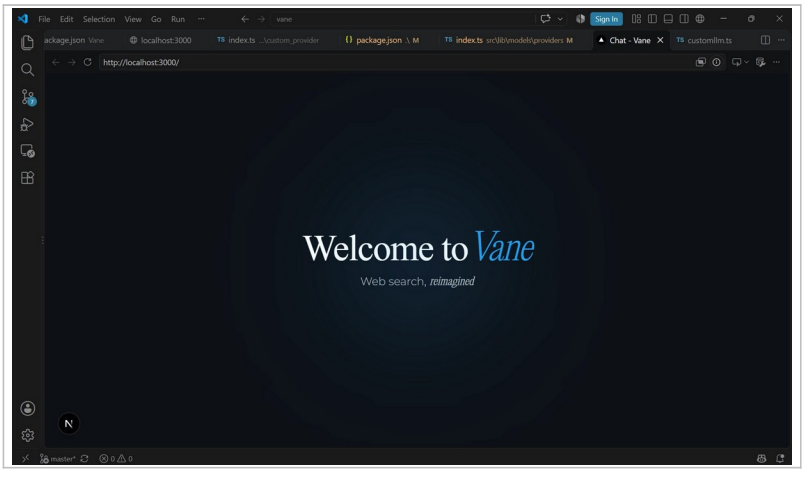
```
npm run dev
```

2. Vane starts, applies its database migrations and reports “Ready”. Leave this window open while you use Vane. To stop Vane, press Ctrl+C in this window.



npm run dev: Vane is ready on http://localhost:3000 and the database migrations have completed

3. Open `http://localhost:3000` in your browser. The first time, Vane shows its welcome screen and then takes you through the setup in Part 3.



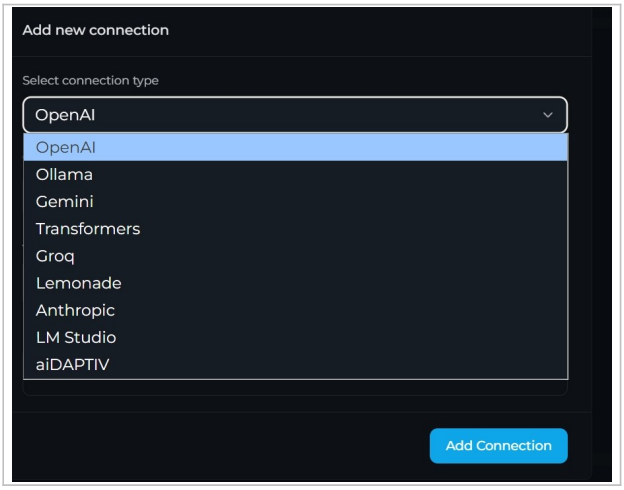
The Vane welcome screen at http://localhost:3000

Part 3: Connecting Vane to aiDAPTIV+

Vane has a built-in **aiDAPTIV** connection type. It sends your questions to the aiDAPTIV llama.cpp server and tells aiDAPTIV+ which folder on the SSD to keep the KV cache in.

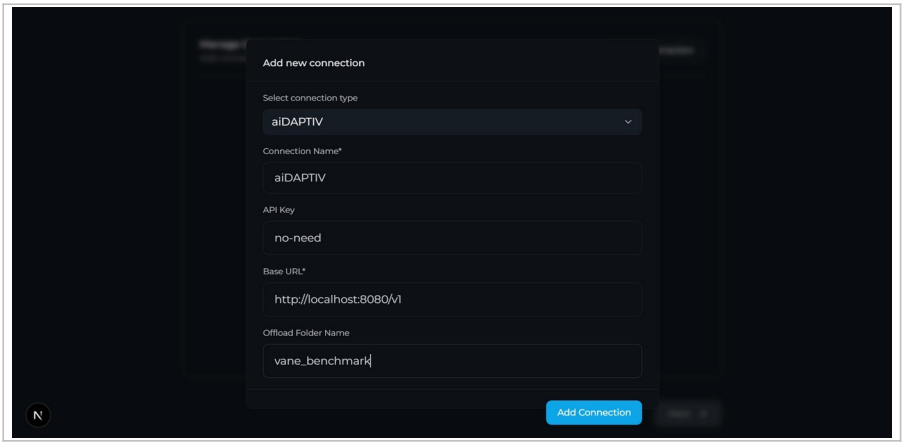
Add an aiDAPTIV connection

1. On the Manage Connections screen, click **Add Connection**.
2. Open the **Select connection type** list and choose **aiDAPTIV**.



The connection types available in Vane, with aiDAPTIV at the bottom of the list

3. Fill in the connection details and click **Add Connection**.



The aiDAPTIV connection details

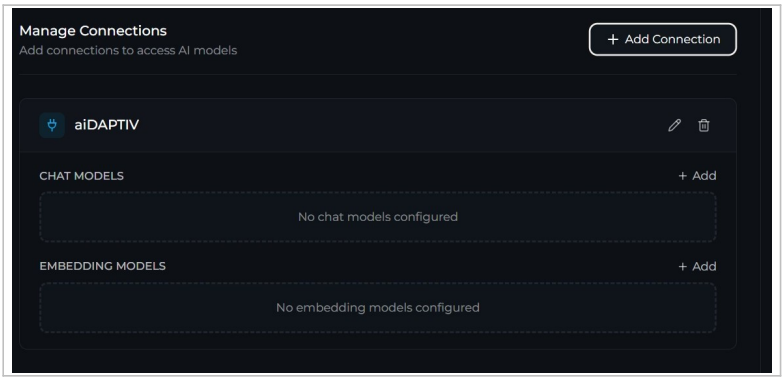
Field	What to enter
Connection type	aiDAPTIV.
Connection Name	Any name you will recognise, for example aiDAPTIV. This name appears in Vane's model list.
API Key	A placeholder such as no-need. The local server does not check it.

Field	What to enter
Base URL	The address of the aiDAPTIV llama.cpp server, `http://localhost:8080/v1` by default.
Offload Folder Name	The subfolder, under your aiDAPTIV NVMe cache root, where the KV-cache files are written, for example vane_benchmark. You will look for this folder again in Part 5.

> **NOTE**
> To keep separate caches, for example one per project or one per test, give each connection its own Offload Folder Name.

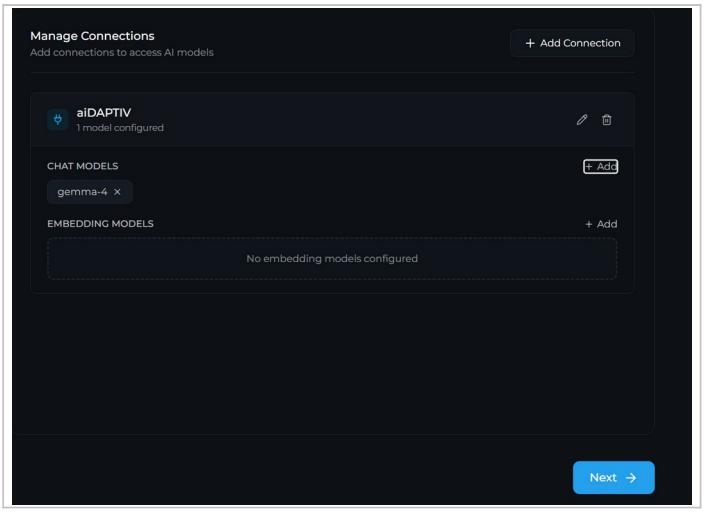
Add the chat model

1. The new aiDAPTIV connection appears with no models yet. Next to **CHAT MODELS**, click **+ Add**.



The aiDAPTIV connection, before any models are added

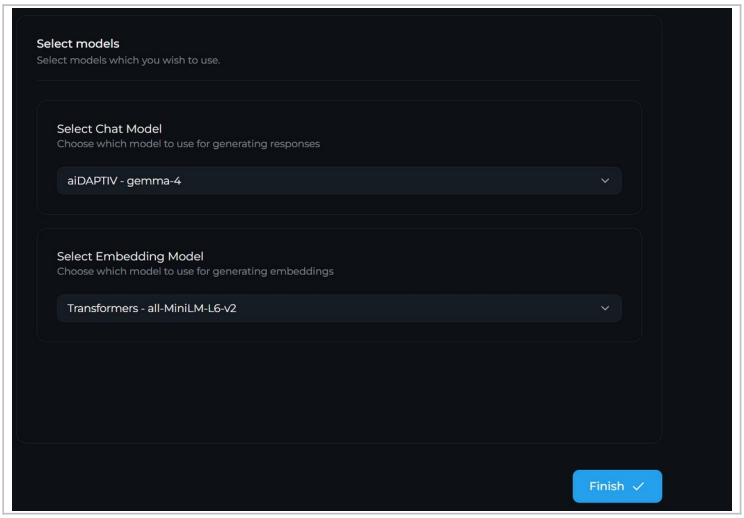
2. Enter the model name and confirm. The safest choice is the exact name your server reports at `/v1/models`. A server that serves a single model answers whatever name it is given, which is why the short name gemma-4 works in our example. The connection now shows “1 model configured”. Click **Next**.



The gemma-4 chat model has been added to the aiDAPTIV connection

Choose the chat and embedding models

1. On the **Select models** screen, choose:
   - **Select Chat Model:** aiDAPTIV - gemma-4 (your aiDAPTIV connection and model).
   - **Select Embedding Model:** Transformers - all-MiniLM-L6-v2, the default.
2. Click **Finish**.



The chat model comes from aiDAPTIV; the embedding model is the default Transformers model

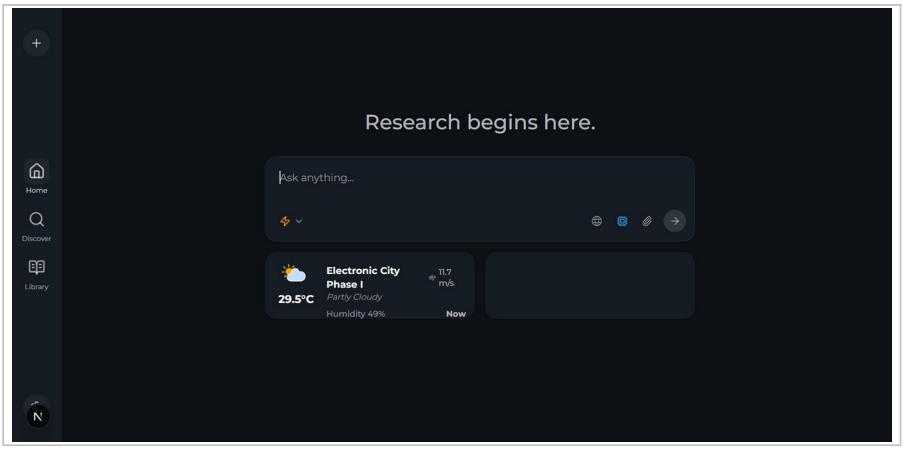
> **NOTE**
>
> The embedding model is mainly used to pick out the most relevant passages from your documents and search results. The default Transformers model, all-MiniLM-L6-v2, runs locally and works well. If you already have your own embedding model, you can add and select it instead.

Restart Vane

1. We recommend restarting Vane once after the first setup. In the PowerShell window running Vane, press Ctrl+C, then start it again:

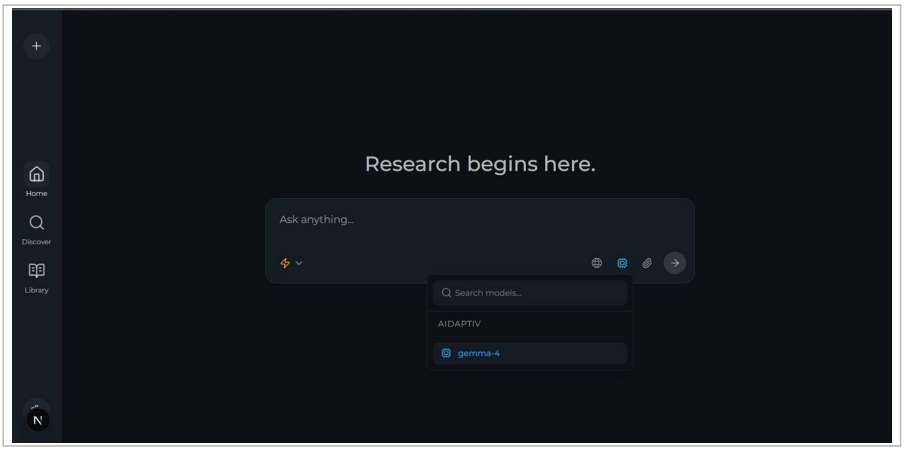
```
npm run dev
```

2. Refresh `http://localhost:3000`. Vane now opens on its home page, “Research begins here.”



The Vane home page after setup

3. Click the model icon (the chip symbol) in the question box. Your aiDAPTIV connection and its gemma-4 model are listed and selected.



The model list, showing the aiDAPTIV connection and the gemma-4 model

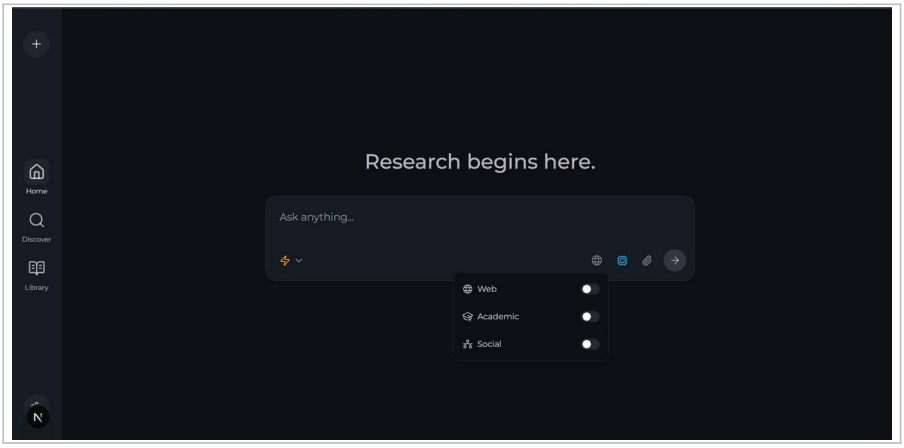
Part 4: Using Vane

The Vane screen

Item	What it does
+ (top left)	Starts a new conversation.
Home	The question box, with a weather card and news cards.
Discover	A feed of current news and articles to read or ask about.
Library	Your past conversations, so you can come back to them later.
Mode (lightning icon)	Chooses how much research Vane does before it answers. The lightning icon is the fastest mode.
Sources (globe icon)	Turns the Web, Academic and Social sources on or off.
Model (chip icon)	Chooses the connection and model that answer the question.
Attach (paper clip icon)	Attaches one or more files for Vane to read.

Choose where Vane searches

Click the globe icon in the question box and switch on the sources you want Vane to search:



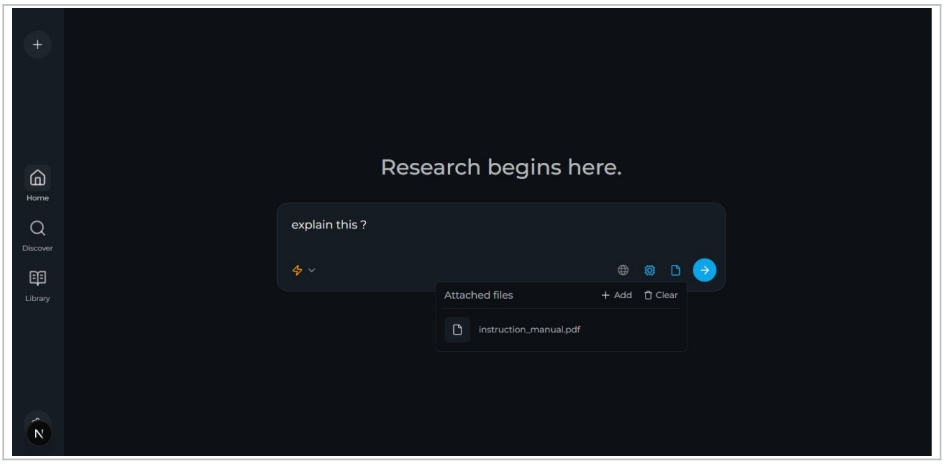
The Web, Academic and Social sources

Source	What Vane searches
Web	The web, through your SearxNG instance.
Academic	Academic and research sites such as arXiv.
Social	Discussion sites such as Reddit.

Type your question in the **Ask anything…** box and press Enter or click the arrow. Vane searches the selected sources, sends the results to your aiDAPTIV+ model and writes an answer with the sources it used.

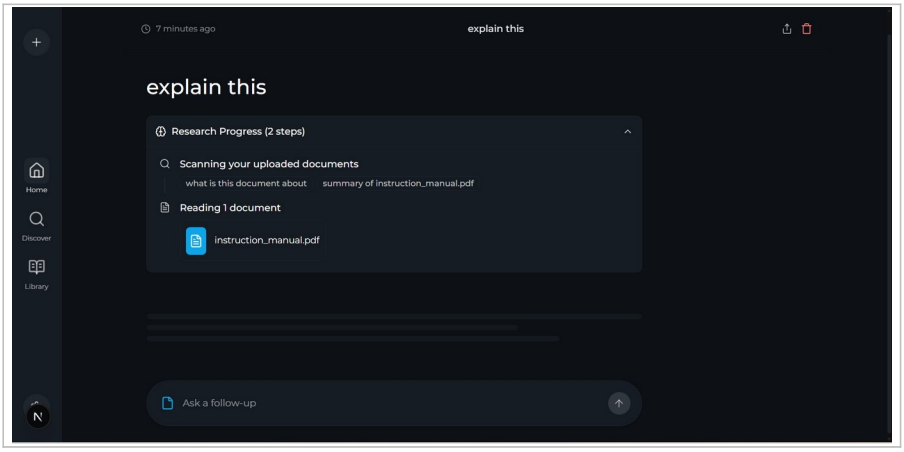
Chat with a document

1. Click the paper clip icon and choose a file. The **Attached files** panel lists it. Use **+ Add** to attach more files or **Clear** to remove them.
2. Type your question, for example “explain this ?”, and press Enter.



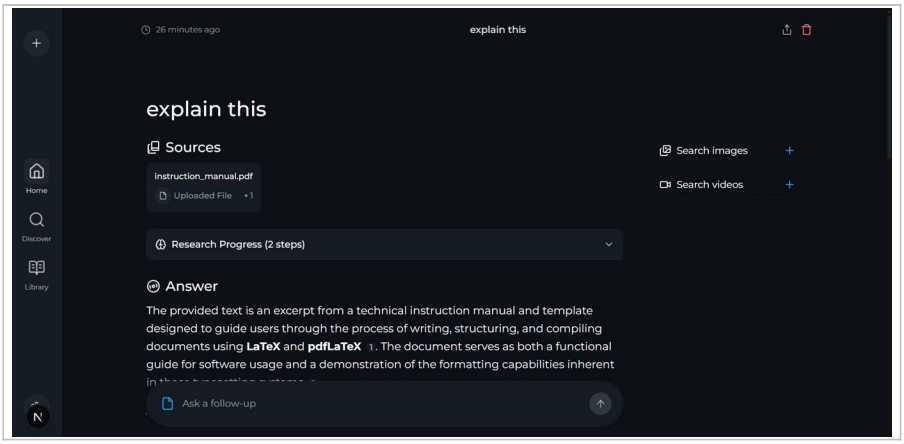
A PDF attached to the question, ready to send

3. Vane shows its **Research Progress** while it works: it scans your uploaded documents, then reads them.



Research Progress: Vane scans and reads the attached document

4. When the model finishes, the answer appears with its **Sources**. Numbers in the text, such as 1, point to the source each statement came from.



The answer, with the uploaded file listed under Sources

From the answer page you can:

- Type in **Ask a follow-up** to keep asking about the same material.
- Use **Search images** and **Search videos** to find related media.
- Use the share icon to share the conversation, or the bin icon to delete it.

> **NOTE**
>
> Follow-up questions about the same document are where aiDAPTIV+ helps most. The document has already been read, so aiDAPTIV+ can reuse its saved notes and the answer starts much sooner. Part 5 shows how to confirm this.

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working

Check the cache folder on disk

After asking Vane at least one question, browse to your aiDAPTIV NVMe cache root (the offload path the server was started with, `D:\nvme0` on our test PC) and find the folder with the Offload Folder Name from the aiDAPTIV connection. Our benchmark connection used vane_benchmark_test. New files there confirm that the KV cache is being written to the aiDAPTIV SSD, not just held in graphics memory.

Check the cache is resumed after a restart

When the model server starts, aiDAPTIV+ reloads the notes it saved on the SSD. Look for these lines near the top of the llama-server log:

```
[MDW][Info][Runtime] Trying to resume KV cache...
[MDW][Info][Runtime] KV cache resume completed
[MDW][Info][Runtime] Resume subfolder: vane_benchmark
[MDW][Info][Runtime] Resume subfolder: vane_benchmark_test
0.25.050.017 I srv  llama_server: server is listening on http://127.0.0.1:8080
```

Server start-up: aiDAPTIV+ resumes the saved KV cache for each Vane offload folder

Confirm a cache hit

1. Attach a long document and ask a question, so that its KV cache is saved.
2. Ask a follow-up question about the same document. You can restart the model server in between to prove the notes survive a restart.
3. Check the llama-server log for the new request and find the **Hit token cnt (CACHE)** value.

This is the log from our test, for the first request Vane sent after the server restart:

```
0.56.932.692 I slot launch_slot_: id  3 | task 0 | processing task, is_child = 0
0.56.933.191 I slot update_slots: id  3 | task 0 | Hit token cnt (GPU): 0
0.57.279.633 I slot update_slots: id  3 | task 0 | Hit token cnt (DRAM): 0
0.57.279.644 I slot update_slots: id  3 | task 0 | Hit token cnt (CACHE): 13392
[MDW][Info][Runtime] Save 0 tokens to folder 'vane_benchmark_test'
0.57.618.442 I slot update_slots: id  3 | task 0 | saved checkpoint to Cache Storage (n_tokens = 13392)
```

Hit token cnt (CACHE): 13392 on the repeat run

A large, non-zero Hit token cnt (CACHE) value is direct proof that those tokens were loaded from the aiDAPTIV NVMe cache instead of being recomputed on the GPU. A closing line such as “Save 0 tokens to folder 'vane_benchmark_test'” confirms that nothing new needed to be cached because it was already there. This lower recompute workload is what produces the faster time to first token on follow-up and overlapping questions.

> **If your second question still shows Hit token cnt (CACHE): 0**
>
> Check that the question was sent through the aiDAPTIV connection (not another connection), that the Offload Folder Name is set on that connection, that the second question is about the same document in the same conversation, and that the cache folder has free space on the aiDAPTIV SSD. Web search results change from search to search, so a repeated web question may reuse less than a repeated document question.

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit

What to measure

The llama-server log reports three figures for each request. These are the ones to note down:

Metric	Where to find it	What it tells you
Hit token cnt (CACHE)	The cache line for the request	How many tokens were loaded from the aiDAPTIV+ cache instead of being recomputed.
TTFT (time to first token)	The prompt eval time line	How long the model took to read the prompt before writing the first word.
TPS (tokens per second)	The eval time line	How quickly the answer was written once it started.

Cache hit (KV reuse) results

From our benchmarking test with a prompt of about 13,400 tokens. The first run had already saved the KV cache for this prompt, the model server was then restarted, and this run reused the cache. Vane sent two requests to the model for the question, and the timing lines for the first are below:

```
0.57.279.644 I slot update_slots: id 3 | task 0 | Hit token cnt (CACHE): 13392
1.15.626.635 I slot print_timing: id 3 | task 0 | prompt eval time = 1106.01 ms / 6 tokens ( 184.34 ms per token, 5.42 tokens per second)
1.15.626.638 I slot print_timing: id 3 | task 0 | eval time = 17237.76 ms / 143 tokens ( 120.54 ms per token, 8.30 tokens per second)
1.15.626.639 I slot print_timing: id 3 | task 0 | total time = 18343.77 ms / 149 tokens
```

Timing lines from the llama-server log for the repeat run

TTFT: 1,106.01 ms • TPS: 8.30 tokens/sec • Hit token cnt (CACHE): 13,392 tokens

Metric	Request 1 (task 0)	Request 2 (task 144)
Hit token cnt (CACHE)	13,392 tokens	12,992 tokens
New prompt tokens processed	6 tokens	11 tokens
TTFT (prompt eval time)	1,106.01 ms	1,488.95 ms
TPS (decode / eval speed)	8.30 tokens/sec	6.34 tokens/sec
Tokens generated	143	2,535

Only a handful of new tokens had to be processed in each request, so the prompt eval time is the time to the first token. All the other tokens came from the aiDAPTIV+ cache. The eval line gives the speed at which the answer was written. The second request wrote a much longer answer, which is why its average speed is a little lower. For comparison, in our CrewAI tests on the same model and class of PC, a prompt of about 12,400 tokens took 4 minutes 38 seconds before the first token without aiDAPTIV+.

Benchmark your own system

1. Set the Offload Folder Name on the aiDAPTIV connection to a new folder name, so the first run starts without a saved cache.
2. Start a new conversation, attach a long document and ask a question. Note the prompt eval time in the server log. This is the first read.
3. Restart the model server, ask a follow-up question about the same document and note the prompt eval time, the eval speed and the Hit token cnt (CACHE) value.
4. For the without-aiDAPTIV+ figure, ask the same question about the same document against a standard llama.cpp server with the same model.

Part 7: Troubleshooting

These are the messages you are most likely to see, and what to do about each one.

Message or symptom	What to do
An error mentioning better-sqlite3, NODE_MODULE_VERSION or “was compiled against a different Node.js version”	The database driver was built for a different Node.js version. Follow “Fixing the better-sqlite3 error” below.
npm warn install-scripts … not yet covered by allowScripts	npm has blocked install scripts. Run `npm install-scripts ls` and approve better-sqlite3, then run `npm rebuild better-sqlite3`.
npm warn deprecated … and a vulnerability summary after npm i	Informational. Vane still works. Avoid `npm audit fix --force`.
npm rebuild fails with compiler errors	Install the Windows C++ build tools and Python, or switch to Node.js 22 (see below). Use the actual error output to decide which is needed.
Port 3000 is already in use	Another program, or another copy of Vane, is using the port. Close it, or stop the other Vane window with Ctrl+C.
The aiDAPTIV model is not listed after setup	Restart Vane (Ctrl+C, then `npm run dev`) and refresh the browser, as described in Part 3.
Questions fail with a connection error	The model server is not running, or the Base URL in the aiDAPTIV connection points at the wrong address or port. Check it with `curl http://localhost:8080/v1/models`.
“Model not found”	Change the chat model name in the aiDAPTIV connection to the exact name the server reports at /v1/models.
Web, Academic or Social searches return no results	Check that your SearxNG instance is running and that Vane is configured with its address.
Hit token cnt (CACHE): 0 on a follow-up question	See the note at the end of Part 5.

Fixing the better-sqlite3 error

The most common problem when installing Vane is a mismatch between the better-sqlite3 native module and the Node.js version, for example a module built for Node.js 22 (`NODE_MODULE_VERSION 127`) running under Node.js 24 (`NODE_MODULE_VERSION 137`). Follow these steps in order.

STEP 1 — CHECK YOUR VERSIONS

Open PowerShell in the Vane folder and check the installed versions:

```
cd C:\Users\<you>\vane
node -v
npm -v
node -p "process.versions.modules"
```

For Node.js 24 the last command should print 137. For Node.js 22 it prints 127.

STEP 2 — TEST WHETHER SQLITE CAN LOAD

```
npm ls better-sqlite3
node -e "const Database = require('better-sqlite3'); const db = new
Database(':memory:'); console.log(db.prepare('SELECT 1 AS result').get()); db.close()"
```

Expected output: `{ result: 1 }`. If you see this, skip to Step 5. Otherwise, continue.

STEP 3 — REBUILD BETTER-SQLITE3 FOR YOUR NODE.JS VERSION

First check whether npm has blocked its install script, and approve the trusted better-sqlite3 install script if your npm version supports it:

```
npm install-scripts --help
```

Then rebuild the module and run the SQLite test from Step 2 again:

```
npm rebuild better-sqlite3 --foreground-scripts
```

If rebuilding fails with compiler errors, you may need the Windows C++ build tools and Python, or a compatible prebuilt binary.

STEP 4 — IF REBUILDING DOES NOT FIX IT, REINSTALL THE LOCAL DEPENDENCIES

This removes only the project's node_modules folder, not your source code or Vane's database. Make sure any local changes are saved first.

```
Remove-Item -Recurse -Force .\node_modules
npm install
```

If npm again reports blocked install scripts, approve them before continuing. The repository also contains a yarn.lock file, so check its README for the intended package manager and use one package manager consistently rather than mixing npm and Yarn. Then run the SQLite test from Step 2 again.

STEP 5 — RUN VANE

```
npm run dev
```

Open `http://localhost:3000` in your browser. If the server starts but reports a new error, troubleshoot that error separately; the native-module problem may be fixed even if another issue remains.

> **STILL NOT WORKING? SWITCH TO NODE.JS 22**
>
> If the problem continues on Node.js 24, switch to Node.js 22 with NVM and reinstall:
>
> `nvm install 22` → `nvm use 22` → delete node_modules → `npm install` → `npm run dev`

Installation checklist

Check	Expected result
Node.js is active	`node -v` reports v24.x.x (or v22.x.x)
Module ABI is correct	`node -p "process.versions.modules"` reports 137 (or 127 on Node.js 22)
better-sqlite3 is installed	`npm ls better-sqlite3` shows the installed package
SQLite test passes	The test returns { result: 1 }
Vane starts locally	`npm run dev` starts without the SQLite error and reports Ready
Model server is reachable	`curl http://localhost:8080/v1/models` lists your model
aiDAPTIV+ is caching	Files appear in the Offload Folder and follow-up questions show a non-zero Hit token cnt (CACHE)